

The Robertson Chemical Kinetics Problem

One fast process, and how it pins the whole computation

Group 5 (names / student IDs to be filled in)

Numerical Analysis of ODEs and PDEs, Project 1, Topic 3

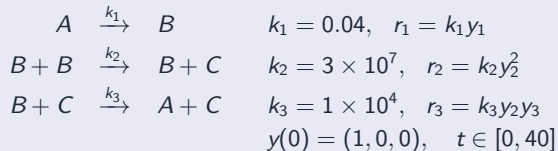
October 4, 2026

What this problem asks: a 1966 stiff benchmark

Problem 3: the Robertson chemical kinetics problem (classic 1966 stiff benchmark)

Three species A, B and C interconvert through three reactions whose rate constants span 0.04 to 3×10^7 ;

$$k_2/k_1 = 7.5 \times 10^8:$$



Mass action: each rate is (rate constant) $\times$ (product of its reactant concentrations). A species gains what its reactions produce and loses what they consume, so the ODEs follow directly:

$$y_1' = -r_1 + r_3, \quad y_2' = +r_1 - r_2 - r_3, \quad y_3' = +r_2$$

The four tasks

- 1 Write Python code that solves this stiff ODE system
- 2 Plot the concentration of A, B, C over time
- 3 Explain what “stiffness” means in this problem
- 4 Compare a non-stiff solver with a stiff solver

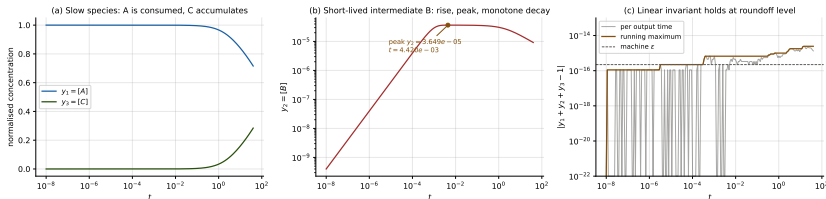
Three things have to be handed in

- the three concentration curves $y_1(t), y_2(t), y_3(t)$
- one specific number: $y_2(40)$
- an explanation: *why this number can be believed*

The plan, in one line

ruler (independent) $\rightarrow$ methods (our own) $\rightarrow$ checks (separate) $\rightarrow$ conclusions (each directly verifiable from the data).

F2 Tight-tolerance reference (Radau, tightened tolerances)



The spike in the middle is B

B jumps up from essentially zero, peaks at $t \approx 0.004$ s, then decays fast: by $t = 40$ s only 9.2×10^{-6} is left.

The two outer curves are A and C

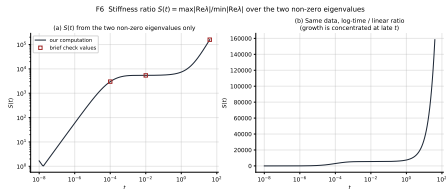
A slides from 1 down to 0.72; C rises from 0 to 0.28. Both move on a **tens-of-seconds** timescale, which has nothing to do with B.

F2: Task 2, the A, B, C concentration curves over time (reference solution, log time axis).

Four conclusions follow, one number each: it really is stiff (predicted 5.90 vs measured 5.42, 8% apart); surviving is not correct (conservation 1.1×10^{-13} yet $\min y_i = -921.8$); implicit is steadier but not cheaper (EE 3.0 vs IE 11 at error 10^{-6}); non-stiff is 100 $\times$ slower (RK45 240,000 evaluations vs BDF 452).

Why it is hard (1): two worlds of different speed, this is “stiff”

Task 3



F6: $S(t) = \text{fastest speed} \div \text{slowest speed}$, taken from the two non-zero eigenvalues of the Jacobian.

Left panel: log-log in time (ours vs reference values); right panel: the same data on a linear scale, where **the gap opens mostly at the end**.

time t	ours	reference
10^{-4}	3.0×10^3	3.0×10^3
10^{-2}	5.4×10^3	5.4×10^3
40	1.58×10^5	1.58×10^5

Two clocks, in one system

B is extremely active: it is eaten within a millionth of a second of being born, and its concentration never gets above about 10^{-5} . So **B owns a fast clock, while A and C own slow ones**. By $t = 40$ the fast clock is **1.58×10^5 times** ahead of the slow one (about 160,000 $\times$). At the start the two differ by only 1.7 $\times$, and *the gap is widest at the very end*.

“Stiff” in plain words

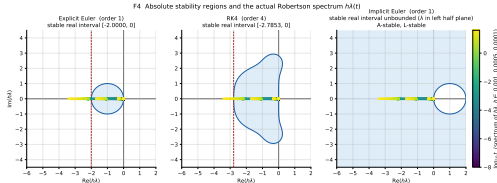
If a system contains **one very fast process** and **one very slow process** at the same time, with a large gap between them, we call it *stiff*.

It does *not* mean “changes violently”. It means “**the fast world and the slow world have to be computed together**”. That is the annoying part.

Why it is hard (2): the consequence, the step size has a ceiling

Why explicit methods have a step ceiling

Explicit Euler and RK4 use only the slope at the left end.
If the step is too large, the error is **amplified** at every step,
so each method has a *safe range* of step sizes.



F4: safe zone from the test equation $y' = \lambda y$: one step multiplies y by $R(z)$, $z = h\lambda$; shaded = $|R(z)| \leq 1$.
Negative real axis: EE safe to $z = -2$, RK4 to $z \approx -2.79$, implicit Euler everywhere.

method	step ceiling (s)	order
Explicit Euler	5.9×10^{-4}	1
RK4	8.2×10^{-4}	4
Implicit Euler	none	1

Where the ceilings come from: fastest eigenvalue of this problem, $|\lambda| = 3392.79 \text{ s}^{-1}$ at $t = 40$.

Ceiling = real-axis edge $\div |\lambda|$: **EE** $2/3392.79 = 5.9 \times 10^{-4}$;
RK4 $2.79/3392.79 = 8.2 \times 10^{-4}$.

A cost floor you cannot get around

With a ceiling of $5.9 \times 10^{-4} \text{ s}$, covering 40 seconds needs at least 6.8×10^4 steps. That is a *floor*: in practice you need even more.

Why the step cannot be relaxed halfway

The ceiling is set by the fastest eigenvalue, which peaks at $t = 40$, so it binds for the whole run. Raise the safe step 5.42×10^{-4} by 15% and by $t = 32.9 \text{ s}$ the concentration reads **-182**: the solution collapses.

Step one: how do we know the ruler can be trusted?

Three independent ways to cross-check

- **Tighten repeatedly:** tighten the solver tolerance again and again, and see at which digit the number stops moving
- **A different author:** compare against two off-the-shelf SciPy solvers (BDF, LSODA)
- **Structural self-check:** the problem has a conservation law of its own; use it as an extra audit

The three routes fail in completely different ways (one through step size, one through implementation, one through algebra), so only if they *all* agree is the answer really trustworthy.

cross-check	digits in agreement
tightening	10.76
vs BDF	10.70
vs LSODA	10.98
lowest	10.76

The largest disagreement between the three is only 2.1×10^{-12} , essentially zero.

Every “error” from now on is a point-by-point comparison with this ruler

The reference solution gives $y_2(40) = 9.185535 \times 10^{-6}$, and it is safe to quote 10 digits.

Step two: three methods, and how each one takes a step

Explicit Euler: only looks at now

$$y_{n+1} = y_n + h f(t_n, y_n)$$

Uses *only* the slope at the left end: one f evaluation, then walk straight along the tangent for the whole step. Nothing looks at what happens *inside* the step. It is cheap, but a large step wrecks it.

RK4: samples the step before committing

$$k_1 = f(t_n, y_n), \quad k_2 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2} k_1), \quad k_3 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2} k_2), \quad k_4 = f(t_n + h, y_n + h k_3)$$

Four slopes (left end, midpoint twice, right end), weighted average $y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$: four evaluations per step, in exchange **4th-order accuracy**, and it can follow the curvature inside the step.

Implicit Euler: looks ahead, must solve an equation

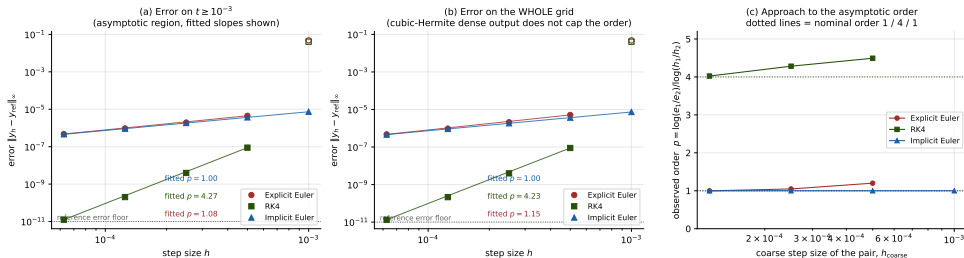
$$y_{n+1} = y_n + h f(t_{n+1}, y_{n+1})$$

The unknown y_{n+1} sits on *both* sides, so every step solves a nonlinear system, by **Newton's method**. That is exactly why we wrote the Jacobian. Each step costs more, but the step itself may be *much* larger: that is the trade.

One real step: from $y_0 = (1, 0, 0)$, $h = 5 \times 10^{-4}$ s, $f(y_0) = (-0.040, +0.040, 0)$; one step gives $y_2 = 2.0 \times 10^{-5}$, and back in f_2 the quadratic term already eats **43%** of it.

Step two (cont.): do our three solvers really have the right order?

F8 Uniform-step convergence, observed order and its limits



F8: error against step size, log-log; the slope of each line *is* the order.

How to read F8: every dot is one run with a fixed h (22 step sizes per method); horizontal axis h , vertical axis the error against the reference ruler. On a log-log plot a p -th-order method is a straight line of slope p , so we do not *claim* the order, we **measure it from the fit**.

method	textbook	measured	fit
EE	1	1.08	0.998
RK4	4	4.27	0.999
IE	1	1.00	1.000

The dotted line is the reference error floor: below it you measure the *ruler*, not the method, so the fit uses $h \geq 10^{-3}$. All three match the textbook: the implementations are **correct**.

Step three: the four checks, one failure each

1 Conservation

What it checks: whether $y_1 + y_2 + y_3$ stays at 1

Why separately: this quantity is **identically 1** in this problem, so it cannot detect an accuracy problem at all (conclusion 2)

2 Non-negativity

What it checks: whether a concentration ever goes negative

Why separately: concentrations are physical; once one turns negative the solution is *meaningless*, however good the other checks look

3 Accuracy

What it checks: point-by-point against the reference, taking the largest error

Why separately: the only one actually measuring precision, and the only one that spots “looks fine, is wrong”

4 Endpoint

What it checks: whether $y_2(40)$ matches the reference

Why separately: it is the number being handed in, but it looks at a single point and **cannot speak for the whole run**

The four checks are **mutually independent**: each can pass or fail on its own. The nastiest combination is “the first three pass, the fourth is way off”. Conclusion 2 nails it down with data.

Step 1 – the model: three reactions as $f(t, y)$

```
K1, K2, K3 = 0.04, 3.0e7, 1.0e4    # rate constants
```

```
def f(t, y):  
    y1, y2, y3 = y  
    r1 = K1 * y1          # A -> B  
    r2 = K2 * y2**2       # B+B -> B+C  
    r3 = K3 * y2 * y3     # B+C -> A+C  
    return [-r1 + r3, r1 - r2 - r3, r2]
```

```
def jac(t, y):            # analytic Jacobian  
    y1, y2, y3 = y  
    b = K3*y3 + 2*K2*y2  
    return [[-K1, K3*y3, K3*y2],  
            [ K1,    -b, -K3*y2],  
            [0.0,  2*K2*y2, 0.0]]
```

The right-hand side is just the **law of mass action**: each rate is (rate constant) $\times$ (product of reactant concentrations), collected species by species. Model plus Jacobian is 14 lines.

Every figure that follows (the concentration curves of F2, the solver shoot-out of F16) comes out of this code.

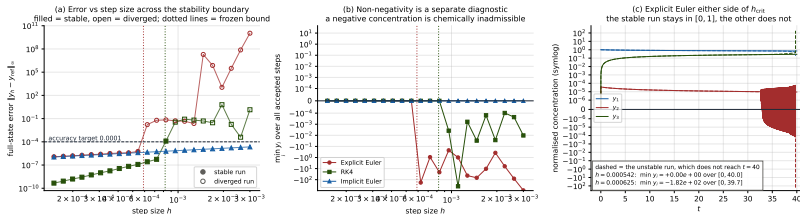
Step 2 – solve: hand it to a stiff solver

```
from scipy.integrate import solve_ivp  
  
sol = solve_ivp(f, (0.0, 40.0), [1.0, 0.0, 0.0],  
                method="BDF",      # stiff solver  
                jac=jac,           # analytic Jacobian  
                rtol=1e-6, atol=1e-9,  
                dense_output=True)  
  
t = sol.t  
y = sol.y      # 3 x N, rows are A, B, C
```

Explicit Euler / RK4 / Implicit Euler are also **written by us**, each under 20 lines in the `src / integrators` modules; this slide only shows the model and the stiff-solver call.

Conclusion 1: it really is stiff, and the bound can be predicted

F7 Frozen-Jacobian estimate vs an empirical step sweep



F7 Each dot is one run of the step sweep (22 step sizes per method).

Left panel: the error. Middle: the lowest concentration reached. Right: two full runs on either side of the limit. A *filled* dot finished, an *open* dot blew up; the dotted lines are the limits we predicted.

How we tested it: first predict the step limit with a short calculation, then run 22 step sizes and see where the solution dies.

The limit comes from **the fastest reaction in the system**, measured at $t = 40$, times each method's safety margin (2.00 for EE, 2.79 for RK4):

EE 5.90×10^{-4} and RK4 8.21×10^{-4} .

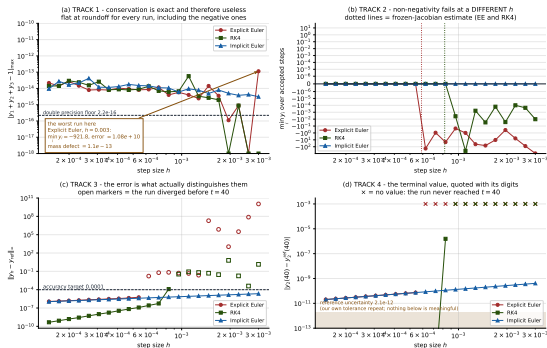
units of 10^{-4} s	EE	RK4
predicted limit	5.90	8.21
largest step that survived	5.42	8.31
first step that blew up	6.25	9.58

The prediction is only 8% off: a short calculation already tells you how small the step has to be.

Conclusion 2: surviving is not correct, and only one check bites

Task 1

F9 Four diagnostics, four verdicts: conservation cannot stand in for accuracy



F9 The same 22 runs, one check per panel: conservation (top left), the lowest concentration (top right), the error (bottom left), the value at $t = 40$ (bottom right). Conservation stays flat in every run, even the collapsed ones; the other three are the checks that react to a bad step.

The most striking single run (explicit Euler, step 3×10^{-3})

conservation = 1.1×10^{-13} (looks perfect)

min concentration = -921.8 (negative!)

error = 1.08×10^{10}

Checking conservation alone would have waved this through.

Why conservation always holds

Add the three equations and **every term cancels in pairs**:

$$\frac{d}{dt}(y_1 + y_2 + y_3) = 0, \text{ so } y_1 + y_2 + y_3 \equiv 1.$$

This is built into the equations themselves, so **every method keeps it automatically**, good run or bad.

So this check measures nothing

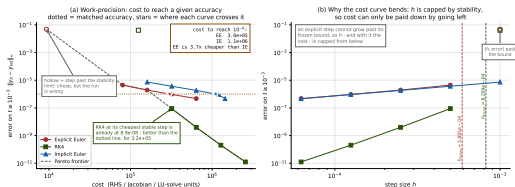
The 10^{-14} in the plot is just the computer's rounding error, not a sign of accuracy.

This check can never fail, no matter how wrong the step is. The other three are the checks that can.

Conclusion 3: implicit is steadier, but not cheaper

Task 4

F12 Cost at matched accuracy - the implicit method is not automatically cheaper



F12

Five step sizes per method; work is counted as one function call, one Jacobian, one solve, each worth 1.

Left: cost to reach a given accuracy; a star marks the 10^{-6} target crossing; *hollow dots* are past the stability limit, cheap but wrong.

Right: why cost cannot keep dropping: the step size is capped by stability.

target error	EE	RK4	IE
10^{-5}	0.80	3.2	1.6
3×10^{-6}	1.1	3.2	3.9
10^{-6}	3.0	3.2	11
10^{-7}	n/a	3.2	n/a

Implicit Euler loses on its own accuracy

Implicit Euler is only 1st order: to reach 10^{-6} it must take very small steps, each $3 \times$ the cost of EE, so in total it is **$3.7 \times$ more expensive**. At 10^{-7} only RK4 gets there. Stability buys a bigger step, not accuracy: it saves no work.

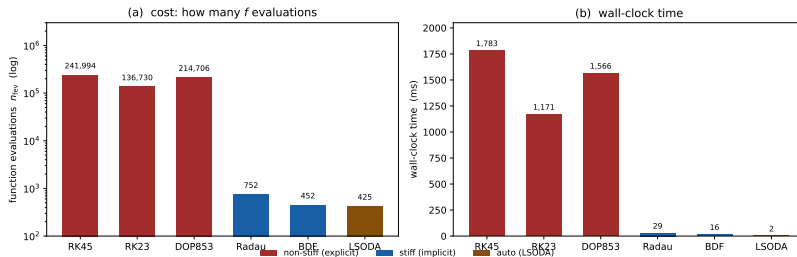
A trap inside implicit methods

Each implicit step solves an equation with Newton's method: if the first guess looks good, the solver **accepts it without iterating**. Loosen the tolerance and the method silently turns back into explicit Euler.

Newton tolerance	error growth	hidden violation
10^{-6}	2.67	$19 \times$
10^{-8}	0.88	none
10^{-10}	1.00	none

10^{-8} is the right tolerance: tighter wastes work; looser and the method is no longer implicit.

F16 scipy solve_ivp: non-stiff vs stiff, same tolerances



F16 The same problem handed to six SciPy `solve_ivp` solvers at the same tolerance, each asked for $y_2(40)$. Left: how many times each solver evaluated the equations; right: how long it took.

The non-stiff solvers cluster near 10^5 evaluations, the stiff ones between 10^2 and 10^3 .

At $\text{rtol} = 10^{-6}$, all six solvers answered $y_2(40)$ correctly to 6 digits:

non-stiff RK45: 240,000 evaluations (1.78 s), DOP853 210,000 (1.57 s);

stiff BDF: 452 (0.016 s), Radau 752 (0.029 s) — the stiff solvers are 100 times faster.

Why are the non-stiff solvers so slow?

Stability forces every explicit step below 10^{-4} s: 40 seconds then takes hundreds of thousands of steps.

An implicit method has no such limit. **Rule of thumb: for a stiff problem, use an implicit (stiff) solver.**

Four conclusions, one number each

Not four parallel findings, but one chain: how to validate → how to measure accuracy → which solver to pick.

conclusion	the number	how to read it
1 it really is stiff	predicted 5.90 / measured 5.42 (10^{-4} s)	8% apart: good enough as a <i>diagnostic</i> , not a theorem
2 surviving is not correct	conservation 1.1×10^{-13} , yet $\min y_i = -921.8$	conservation is structurally exact , so it cannot fail; only pointwise comparison bites
3 implicit is not cheaper	at 10^{-6} error: EE 3.0, IE 11	stability is bought, not accuracy: $3.7\times$ the cost at 10^{-6}
4 non-stiff is $100\times$ slower	RK45 240,000 evaluations, BDF 452	explicit is pinned by the stability bound, implicit is not, so the choice is the result

The one number we have to hand in

$y_2(40) = 9.185535 \times 10^{-6}$, quotable to **10 digits**. The peak 3.65×10^{-5} (at $t \approx 0.004$ s) is 4 times larger than the endpoint, so we report both.

Reproducibility and AI transparency

One command recomputes everything

`python run_all.py` (about 6 minutes)

reference solution, eigenvalues, stability, integrators, step sweep, diagnostics, Newton sweep, adaptivity, figures: 10 stages, fully automatic.

- not a single number is typed by hand in the results
- SciPy is used *only* for the reference cross-check; EE / RK4 / IE / Newton are **all implemented by us**
- 148 tests, all green

Note on AI use

This project was completed with AI assistance; the full log is in `appendix/ai_transparency_log.md`, recording entry by entry “what was wrong, how it was found, what it became”: 18 entries in all. Typical examples: a wrongly written RK4 stability polynomial, a transposed Jacobian check matrix, and the mistaken belief that a loose tolerance “only costs accuracy” (in fact a loose tolerance quietly changes the method). All of these were fixed and covered by regression tests.